

Практическое задание 3

Реализация когнитивного прототипа AI-системы

Emotion Recognition System — распознавание эмоций по выражению лица

Студенты: Akhmedov Adilet, Rymgali Assanali, Faizolla Amanbek

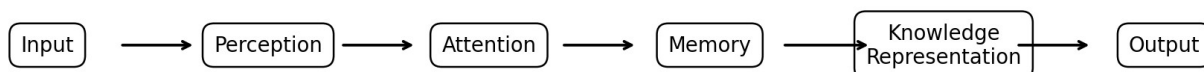
Группа: AAI-2501M

2026

1. Связь с Assignment 2 и цель прототипа

Assignment 2 описывал Emotion Recognition System как систему, которая получает изображение/видео лица, извлекает признаки выражения, отбирает надёжные фрагменты, использует память и выдаёт метку эмоции с уровнем уверенности. В Assignment 3 эта архитектура реализована в работающем программном прототипе.

Цель: реализовать единый сценарий Input → Perception → Attention → Memory → Knowledge Representation → Output и продемонстрировать его минимум на трёх примерах.



Docker Compose: Streamlit app + PostgreSQL + data/training profiles

2. Технологический стек и Docker Compose

Компонент	Технология	Назначение
Perception	OpenCV + PyTorch CNN	Face ROI, grayscale 48×48, 7-class probabilities
Attention	Quality + confidence thresholds	Отбор надёжных кадров и relevance score
Short-term Memory	Python deque	История последних результатов текущей session
Long-term Memory	PostgreSQL 16	INSERT/SELECT результатов и thresholds между запусками
Knowledge Representation	NetworkX + if-then rules	Сущности, связи, правила и объяснение решения
UI / Demo	Streamlit	Webcam/upload, визуализация модулей и результата
Infrastructure	Docker Compose	Единый воспроизводимый запуск app + DB + data/train profiles

Основной запуск выполняется через Docker Compose. Профиль data скачивает datasets и формирует общий индекс; профиль train обучает CNN; обычный docker compose up запускает Streamlit и PostgreSQL. Отдельный notebook-профиль доступен для демонстрации Jupyter внутри того же окружения.

3. Наборы данных

Проект использует три внешних набора данных по распознаванию выражений лица. Они не включены в архив проекта из-за размера и условий распространения; скачивание выполняет официальный Python-клиент KaggleHub во время setup.

Dataset	Источник	Используемые классы	Роль
FER-2013	Kaggle: msambare/fer2013	7 эмоций	Основной in-the-wild baseline; 48×48 grayscale
CK+ / CK+48	Kaggle: shawon10/ckplus	anger, disgust, fear, happy, sad, surprise; contempt	Лабораторные выражения, дополнительное

		игнорируется	разнообразие
RAF-DB	Kaggle: shuvoalok/raf-db-dataset	7 базовых эмоций	Более реалистичные RGB изображения; приводятся к grayscale 48×48

Нормализация labels приводит источники к общей схеме: angry, disgust, fear, happy, neutral, sad, surprise. Для учебного CPU-запуска prepare_dataset.py по умолчанию ограничивает максимум 1500 изображений на пару source/class; ограничение можно отключить. Разбиение train/val/test выполняется детерминированно с seed=42.

4. Реализация обязательных когнитивных модулей

4.1 Perception

Входом является изображение с камеры Streamlit или JPG/PNG. OpenCV ищет лицо Haar Cascade. Если dataset уже содержит cropped face и detector не находит лицо, используется center-crop fallback. ROI переводится в grayscale 48×48 и подаётся в CNN. Результат модуля: probabilities 7 классов, top emotion, confidence, brightness, sharpness и quality score.

Обоснование: OpenCV обеспечивает лёгкий и воспроизводимый preprocessing, а CNN соответствует технологическому стеку Assignment 2 и напрямую выдаёт вероятностное представление выражения лица.

4.2 Attention / Selection

Система не доверяет любому прогнозу CNN. Кадр принимается только если $quality \geq 0.45$ и $confidence \geq 0.55$. Для ранжирования вычисляется $relevance = 0.45 \times quality + 0.55 \times confidence$. Поэтому на демонстрации можно сравнить high-relevance и low-relevance кадр и объяснить критерий отбора.

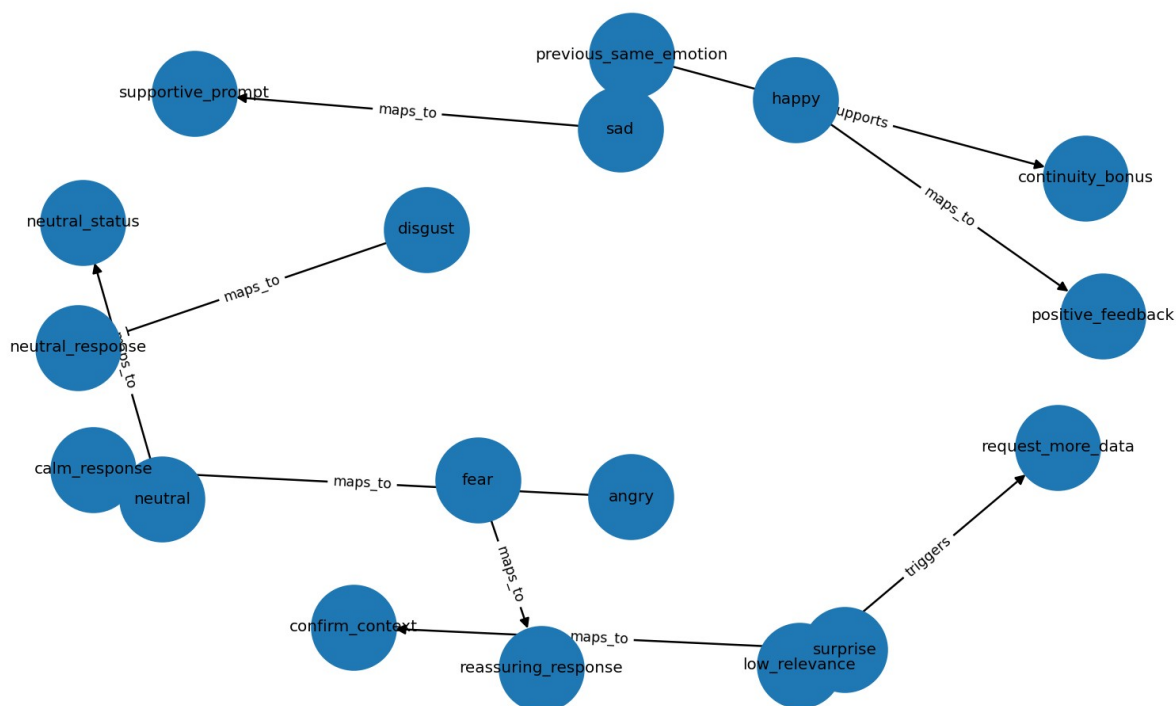
4.3 Memory

Кратковременная память — deque(maxlen=5) для каждой session_id. В неё попадают probabilities и факт принятия/отклонения кадра. Accepted predictions усредняются и сглаживают текущий результат. Долговременная память реализована PostgreSQL: таблица predictions сохраняет session_id, emotion, confidence, quality и relevance; таблица settings хранит thresholds. Перед решением выполняется SELECT предыдущего результата, после надёжного решения — INSERT.

Влияние памяти: если предыдущий сохранённый результат той же session совпадает с текущей emotion, rule R2 добавляет небольшой continuity bonus +0.05. Memory не может переопределить отклонение Attention.

4.4 Knowledge Representation

Знания представлены графом NetworkX. Типы сущностей: Emotion, Action, Condition, MemoryFact. Базовые связи: emotion → maps_to → action; low_relevance → triggers → request_more_data; previous_same_emotion → supports → continuity_bonus.



- R1: если Attention отклонил вход → insufficient_data / request_more_data.
- R2: если previous stored emotion совпадает → +0.05 continuity bonus.
- R3: если adjusted confidence < 0.60 → uncertain.
- R4: accepted emotion → соответствующее action (например happy → positive_feedback).

5. Интеграция и три демонстрационных сценария

Пример	Input	Perception	Attention	Memory / Knowledge	Output
1. Clear	Чёткое лицо из test split	7 probabilities + high quality	Accepted	Запись в PostgreSQL; R4	Emotion + confidence + action
2. Mild degradation	То же лицо, mild blur/darken	Confidence/quality ниже	Ожидается accepted	SELECT previous + STM smoothing + R2	Стабилизированный результат
3. Low quality	Сильный blur + darken	Прогноз может существовать, quality низкий	Rejected	R1 имеет приоритет над Memory	insufficient_data

CLI-команда `docker compose exec app python scripts/run_demo.py` воспроизводит эти три сценария на одном test-изображении. В Streamlit те же этапы показываются интерактивно.

6. Пошаговый запуск

1. `cp .env.example .env`
2. `docker compose --profile data run --rm dataset-downloader`
3. `docker compose --profile train run --rm trainer`
4. `docker compose up --build`

5. Открыть `http://localhost:8501`

6. `docker compose exec app python scripts/run_demo.py`

7. `docker compose down`

Если Kaggle требует авторизацию, в `.env` указывается `KAGGLE_API_TOKEN`. KaggleHub официально поддерживает `dataset_download` и токен через `environment variable`.

7. Приватность, ограничения и воспроизводимость

Система не должна сохранять сырые изображения лиц, видео, biometric templates или идентифицирующие данные без явного согласия пользователя. В prototype долговременная память хранит только metadata прогноза. Для production нужны access control, encryption, retention policy и механизм удаления.

Ограничение интерпретации: система классифицирует наблюдаемое выражение лица, а не устанавливает внутреннее эмоциональное состояние человека как объективный факт. Ошибки возможны из-за освещения, позы, окклюзии, культурных различий и domain shift между datasets.

Воспроизводимость обеспечивается Dockerfile, docker-compose.yml, фиксированным seed=42, явными dataset handles, общим preprocessing и сохранением model metrics в JSON.

8. Вклад участников

Перед сдачей группа должна заменить этот абзац на фактическое распределение работы. Рекомендуемый формат: участник → конкретные модули/код/документация/демонстрация. Не следует указывать вклад, который участник фактически не выполнял.

9. Источники данных и ПО

- FER-2013: <https://www.kaggle.com/datasets/msambare/fer2013>
- CK+: <https://www.kaggle.com/datasets/shawon10/ckplus>
- RAF-DB: <https://www.kaggle.com/datasets/shuvoalok/raf-db-dataset>
- KaggleHub: <https://github.com/Kaggle/kagglehub>
- PyTorch: <https://pytorch.org/>
- OpenCV: <https://opencv.org/>
- PostgreSQL: <https://www.postgresql.org/>
- Docker Compose: <https://docs.docker.com/compose/>